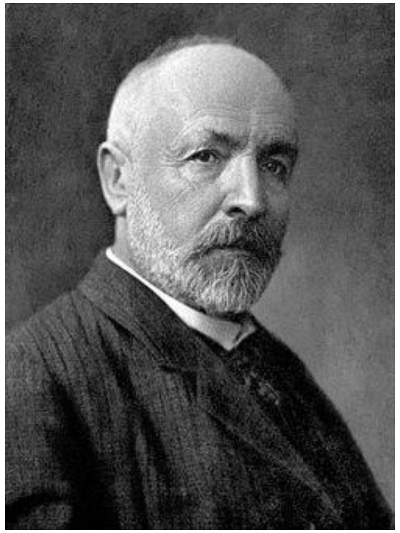
Introduccion al Curso

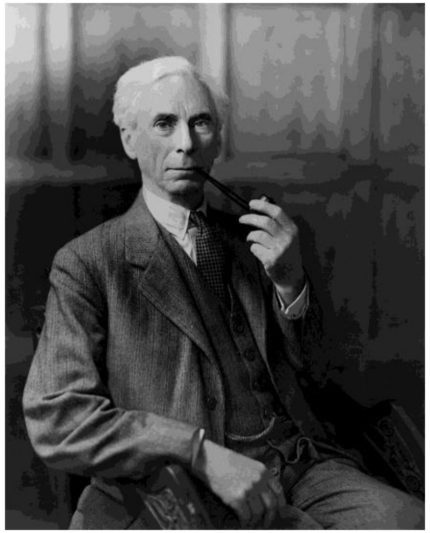
Enzo Alda

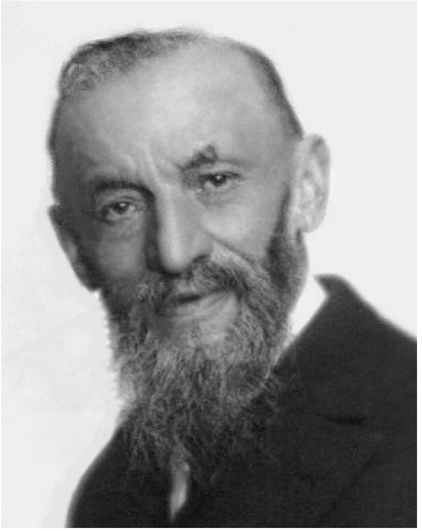
Simon Bolivar University

UNIVERSIDAD SIMÓN BOLÍVAR

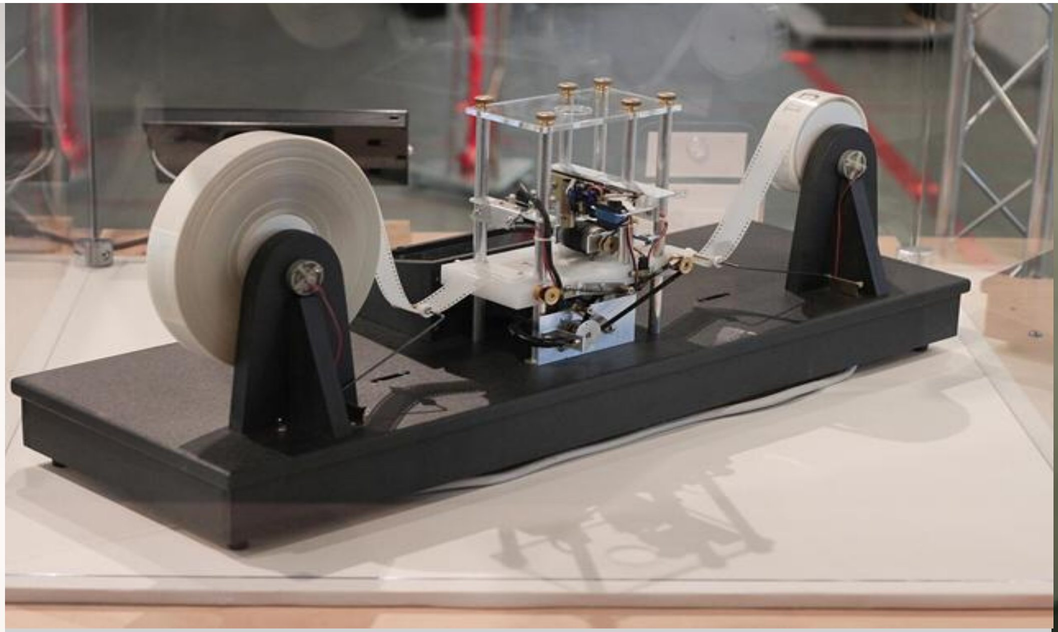
From Euler to the Foundational Crisis of Mathematics to Kurt Gödel incompleteness theorems







Alonzo Church & Alan Turing



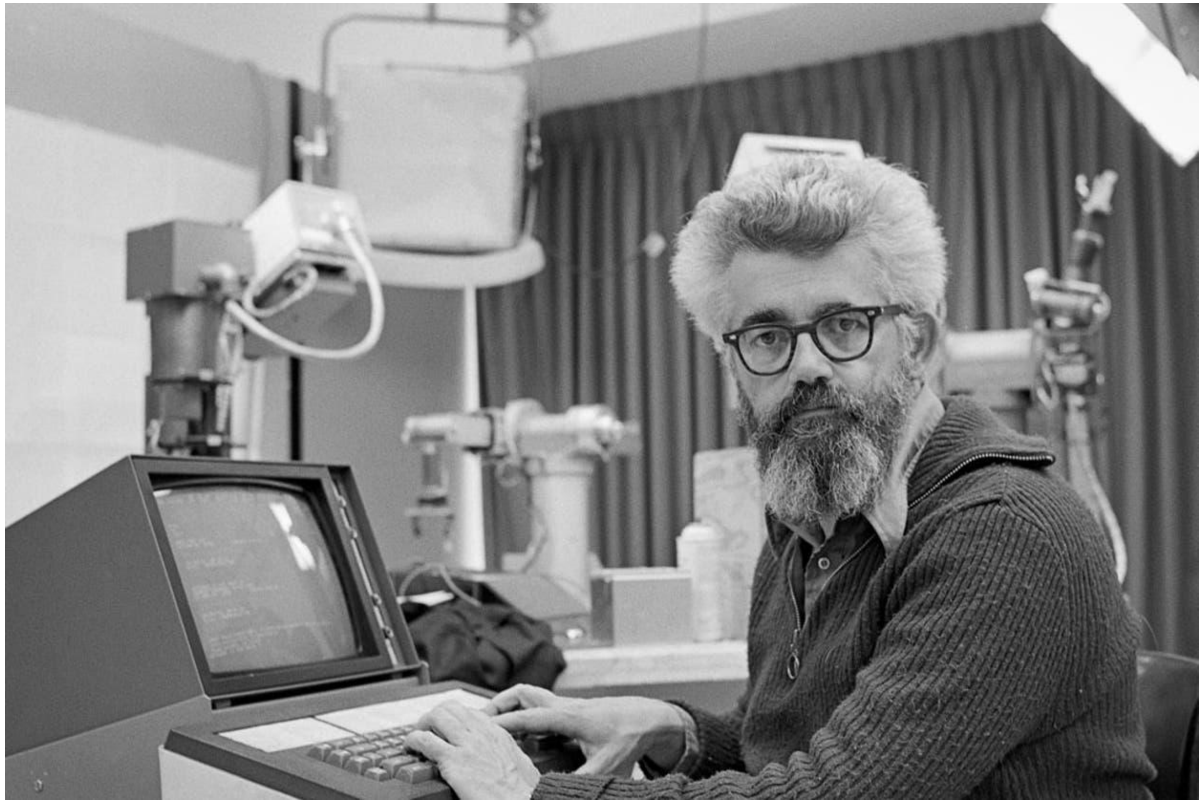
Lambda Cálculo:

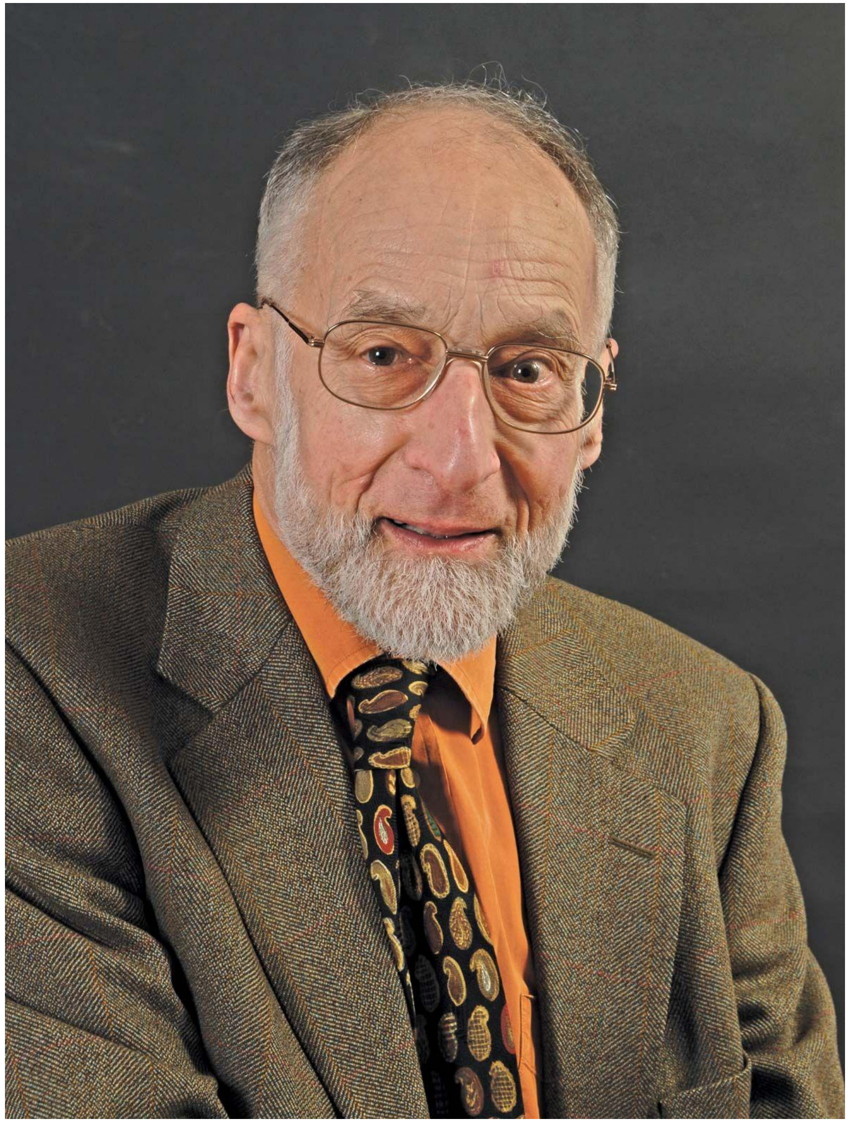
- Un símbolo *x* es un λ-term
- *(λx.t)* es un λ-term llamado "abstracción funcional"
- *(t s)* es un λ-term llamado "aplicación"

LISP - John McCarthy

- (define (list-of-squares n)
- (let loop ((i n) (res '()))
- (if (< i 0)
- res
- (loop (- i 1) (cons (* i i) res)))))

- (list-of-squares 9)
- ===> (0 1 4 9 16 25 36 49 64 81)





Robin Milner

- LCF
- ML
- CCS
- π-calculus
- Hindley-Milner algorithm (inferencia de tipos)

David Turner

Lazy Evaluation – Evaluación Perezosa

Combinator Graph Reduction

Polymorphic Types – Tipos Polimorfos

SASL

KRC

Miranda

From "Research Topics in Functional Programming" ed. D. Turner, Addison-Wesley, 1990, pp 17–42.

Why Functional Programming Matters

John Hughes
The University, Glasgow

Abstract

As software becomes more and more complex, it is more and more important to structure it well. Well-structured software is easy to write and to debug, and provides a collection of modules that can be reused to reduce future programming costs. In this paper we show that two features of functional languages in particular, higher-order functions and lazy evaluation, can contribute significantly to modularity. As examples, we manipulate lists and trees, program several numerical algorithms, and implement the alpha-beta heuristic (an algorithm from Artificial Intelligence used in game-playing programs). We conclude that since modularity is the key to successful programming, functional programming offers important advantages for software development.

The **ZenSheet**™ Project

Una expedición teórica y practica para unificar las hojas de cálculo con ambientes modernos de programación

- Lakebolt Research fue fundada el año 2015, al haber logrado prueba clara del concepto de ZenSheet
- SPLASH 2017 (Vancouver) – LIVE 2017: ZenSheet demo - Mónica Figuera "*ZenSheet Studio*" publicación en la ACM DL
- Lambda Days 2020 (Krakow) – demo de ZenSheet (45 min) usando la nueva implementación de Lilly, de Javier López
- ICICT 2021 (Shanghái – remote) – "*Towards Wide-Spectrum Spreadsheet Computing*" publicación en IEEE Xplore
- ISEC 2023 (Maryland) – "An Expression-Oriented Approach to Programming Education" publicación en IEEE Xplore
- ICFP 2024 (Milán) – Daniel Pinto logra el podio por su trabajo en la semántica operacional de Zilly

La Universidad Simón Bolívar, en particular el profesor Baralt y el profesor Flaviani, han sido un gran apoyo ...
... y las contribuciones de estudiantes y egresados de la USB ha permitido llevar ZenSheet más lejos de lo esperado

Lakebolt Research

UNIVERSIDAD SIMÓN BOLÍVAR

(optional ZenSheet tour in the next 5 slides) =>

SPLASH VANCOUVER 2017 OCTOBER 22–27

Attending Program Tracks Organization Search Series Sign in Sign up

SPLASH 2017 (series) / LIVE 2017 (series) / LIVE 2017 /

ZenSheet: a live programming environment for reactive computing

Who *Enzo Alda, Monica Figuera*

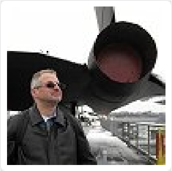
Track LIVE 2017

When **Tue 24 Oct 2017 15:30 - 15:50 at Regency D** - Winter

Abstract We introduce ZenSheet: an experimental live programming environment for reactive computing. We emphasize the most relevant design and implementation choices, and provide a glimpse about future work. The ZenSheet project aims to generalize spreadsheets in an intuitive way. It implements a superset of the core functionality of traditional spreadsheets, adding concepts from modern programming languages. The end purpose is to make spreadsheet computing valuable to an even wider audience spectrum.

File attachments

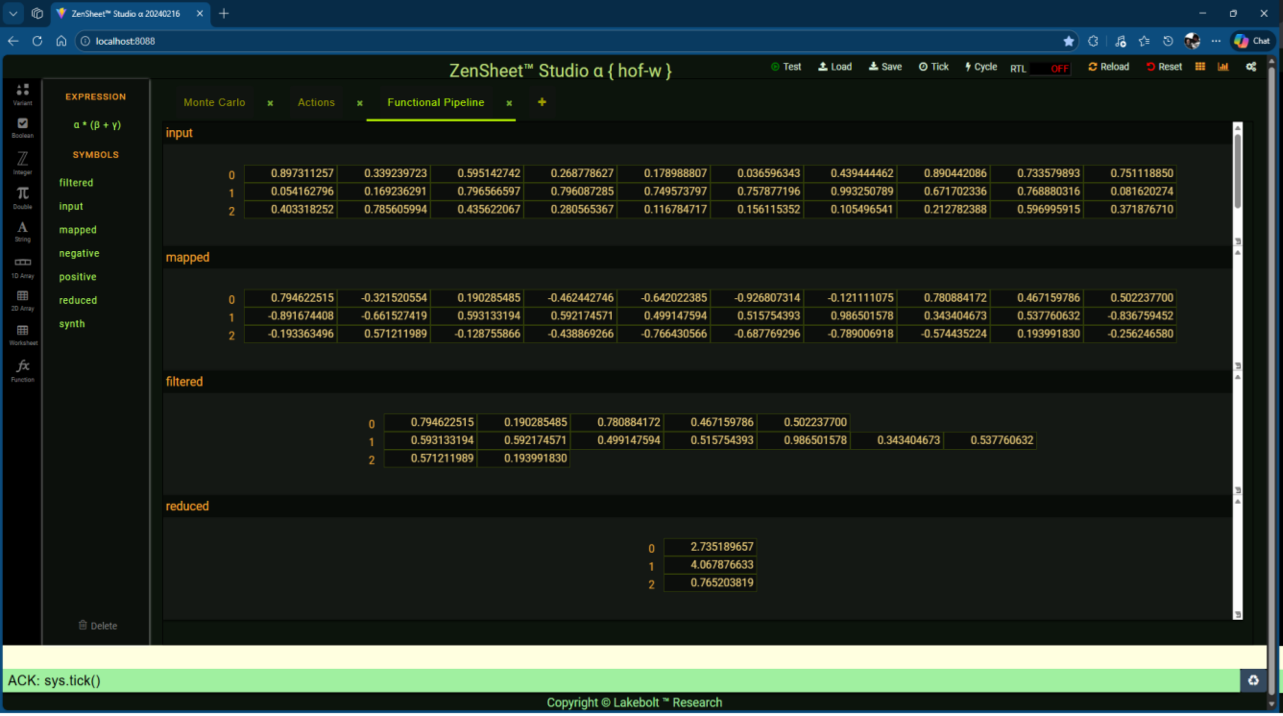
PDF Preprint (ZenSheet.pdf)	475KiB



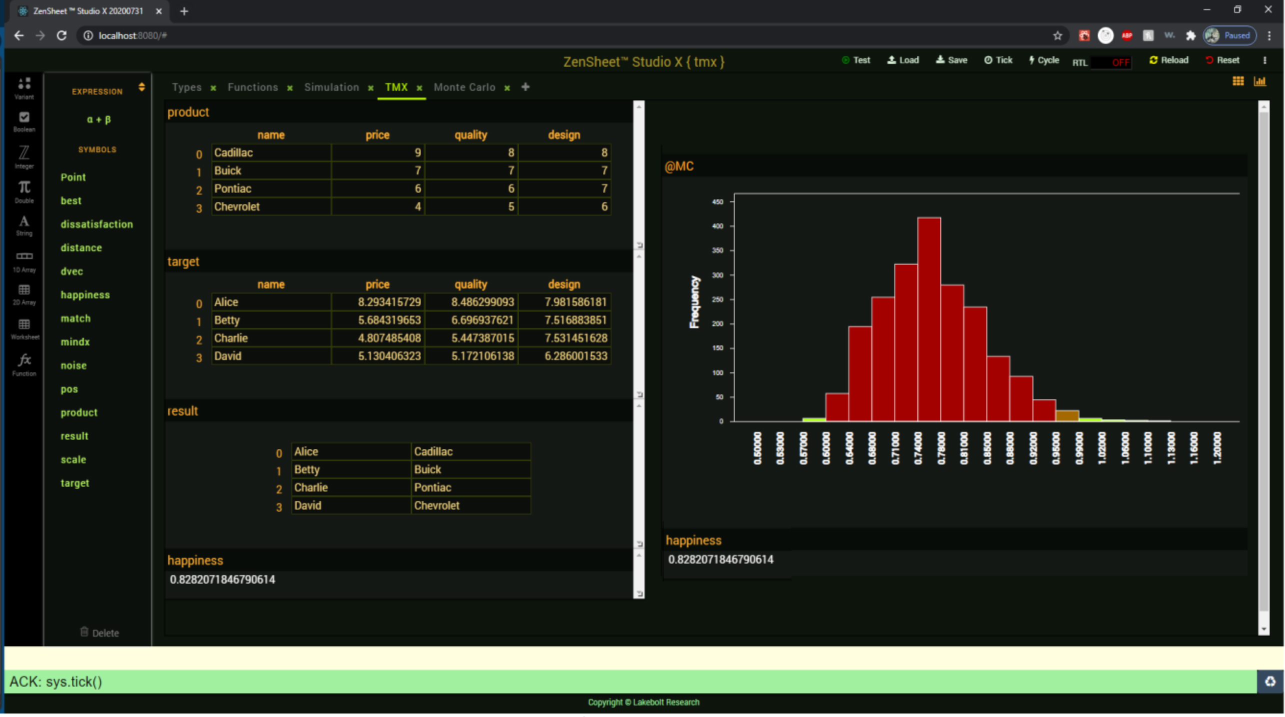
Enzo Alda
Lakebolt Research
United States

Monica Figuera
Universidad Simón Bolívar

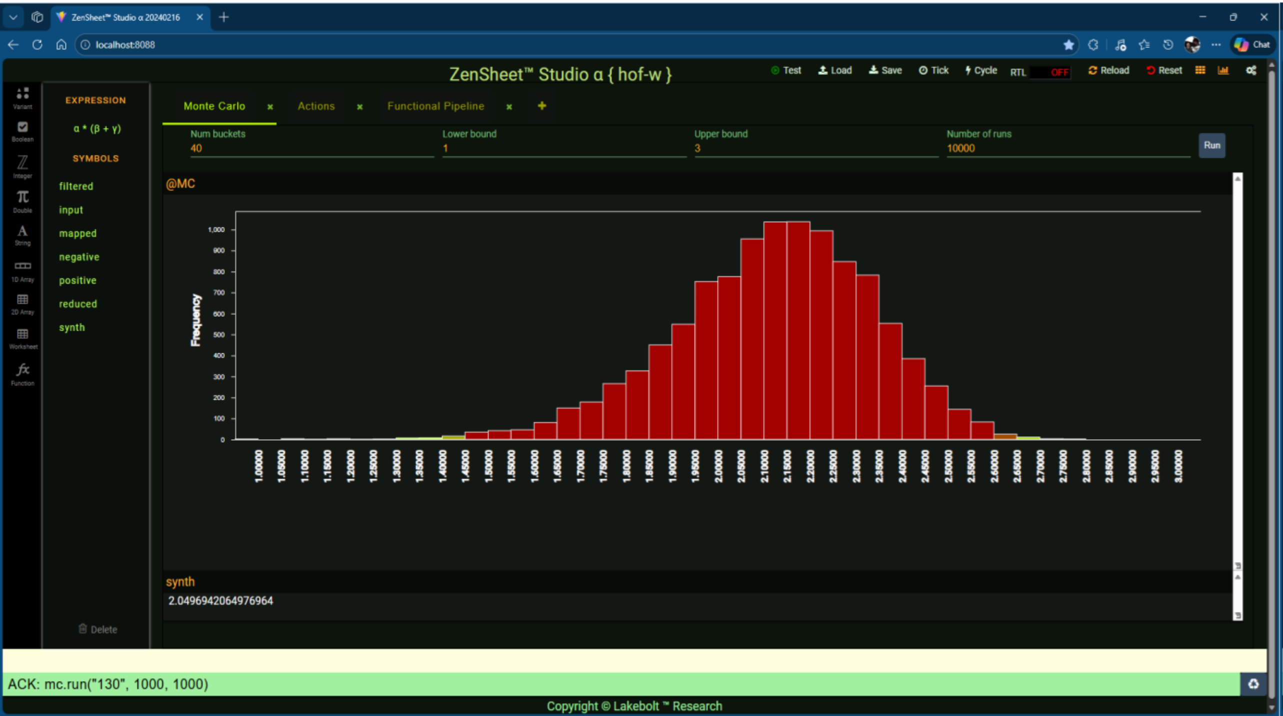
Web Supplement (+ videos) Web Supplement (+ videos)



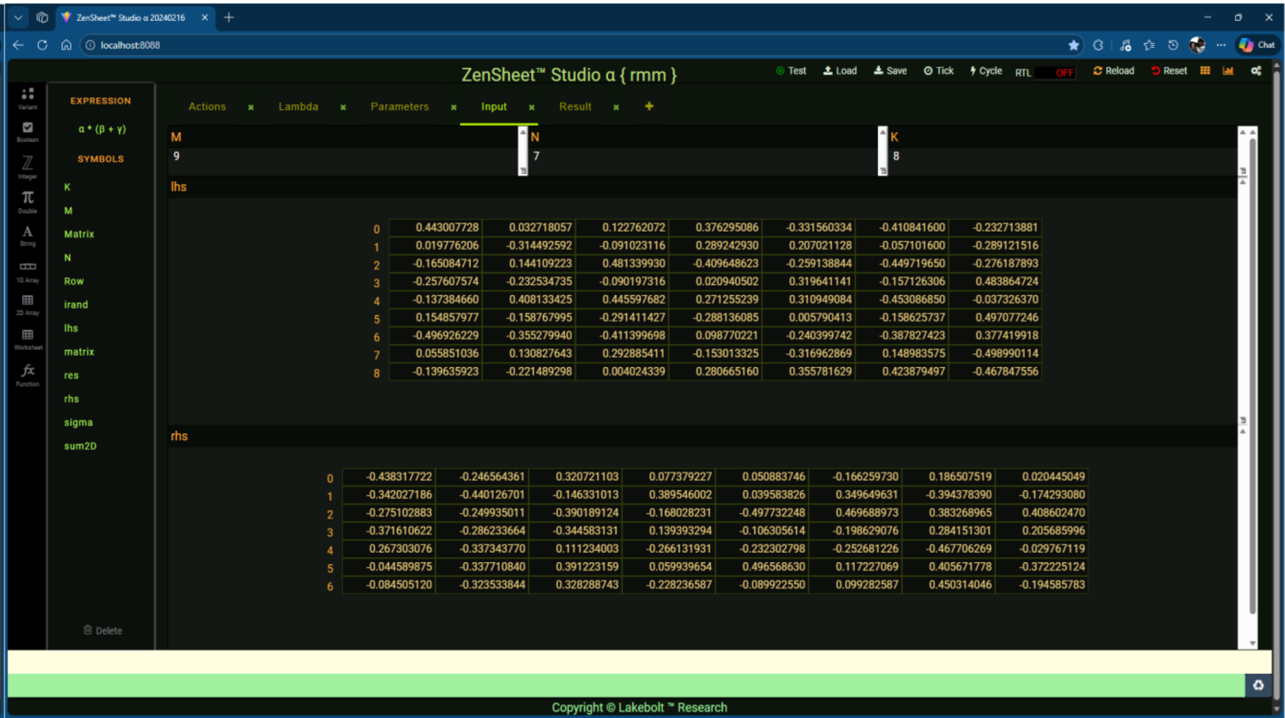
ZenSheet™ Studio α { hof-w }
Test Load Save Tick Cycle RTL OFF Reload Reset
EXPRESSION
α * (β + γ)
SYMBOLS
filtered
input
mapped
negative
positive
reduced
synth
Delete
Monte Carlo
Actions
Functional Pipeline
input
0 0.897311257 0.339239723 0.595142742 0.268778627 0.178988807 0.036596343 0.439444462 0.890442086 0.733579893 0.751118850
1 0.054162796 0.169236291 0.796566597 0.796087285 0.749573797 0.757877196 0.993250789 0.671702336 0.768880316 0.081620274
2 0.403318252 0.785605994 0.435622067 0.280565367 0.116784717 0.156115352 0.105496541 0.212782388 0.596995915 0.371876710
mapped
0 0.794622515 -0.321520554 0.190285485 -0.462442746 -0.642022385 -0.926807314 -0.121111075 0.780884172 0.467159786 0.502237700
1 -0.891674408 -0.661527419 0.593133194 0.592174571 0.499147594 0.515754393 0.986501578 0.343404673 0.537760632 -0.836759452
2 -0.193363496 0.571211989 -0.128755866 -0.438869266 -0.766430566 -0.687769296 -0.789006918 -0.574435224 0.193991830 -0.256246580
filtered
0 0.794622515 0.190285485 0.780884172 0.467159786 0.502237700
1 0.593133194 0.592174571 0.499147594 0.515754393 0.986501578 0.343404673 0.537760632
2 0.571211989 0.193991830
reduced
0 2.735189657
1 4.067876633
2 0.765203819
ACK: sys.tick()
Copyright © Lakebolt ™ Research



ZenSheet™ Studio X { tmx }
Test Load Save Tick Cycle RTL OFF Reload Reset
EXPRESSION
α + β
SYMBOLS
Point
best
dissatisfaction
distance
dvec
happiness
match
mindx
noise
pos
product
result
scale
target
Delete
Types
Functions
Simulation
TMX
Monte Carlo
product
name price quality design
0 Cadillac 9 8 8
1 Buick 7 7 7
2 Pontiac 6 6 7
3 Chevrolet 4 5 6
target
name price quality design
0 Alice 8.293415729 8.486299093 7.981586181
1 Betty 5.684319653 6.696937621 7.516883851
2 Charlie 4.807485408 5.447387015 7.531451628
3 David 5.130406323 5.172106138 6.286001533
result
0 Alice Cadillac
1 Betty Buick
2 Charlie Pontiac
3 David Chevrolet
happiness
0.8282071846790614
@MC
Frequency
happiness
0.8282071846790614
ACK: sys.tick()
Copyright © Lakebolt Research



ZenSheet™ Studio α { hof-w }
Test Load Save Tick Cycle RTL OFF Reload Reset
EXPRESSION
α * (β + γ)
SYMBOLS
filtered
input
mapped
negative
positive
reduced
synth
Delete
Monte Carlo
Actions
Functional Pipeline
Num buckets
40
Lower bound
1
Upper bound
3
Number of runs
10000
Run
@MC
Frequency
synth
2.0496942064976964
ACK: mc.run("130", 1000, 1000)
Copyright © Lakebolt ™ Research



ZenSheet™ Studio α { rmm }
Test Load Save Tick Cycle RTL OFF Reload Reset
EXPRESSION
α * (β + γ)
SYMBOLS
K
M
Matrix
N
Row
irand
lhs
matrix
res
rhs
sigma
sum2D
Delete
Actions
Lambda
Parameters
Input
Result
M
9
N
7
K
8
lhs
0 0.443007728 0.032718057 0.122762072 0.376295086 -0.331560334 -0.410841600 -0.232713881
1 0.019776206 -0.314492592 -0.091023116 0.289242930 0.207021128 -0.057101600 -0.289121516
2 -0.165084712 0.144109223 0.481339930 -0.409648623 -0.259138844 -0.449719650 -0.276187893
3 -0.257607574 -0.232534735 -0.090197316 0.020940502 0.319641141 -0.157126306 0.483864724
4 -0.137384660 0.408133425 0.445597682 0.271255239 0.310949084 -0.453086850 -0.037326370
5 0.154857977 -0.158767995 -0.291411427 -0.288136085 0.005790413 -0.158625737 0.497077246
6 -0.496926229 -0.355279940 -0.411399698 0.098770221 -0.240399742 -0.387827423 0.377419918
7 0.055851036 0.130827643 0.292885411 -0.153013325 -0.316962869 0.148983575 -0.498990114
8 -0.139635923 -0.221489298 0.004024339 0.280665160 0.355781629 0.423879497 -0.467847556
rhs
0 -0.438317722 -0.246564361 0.320721103 0.077379227 0.050883746 -0.166259730 0.186507519 0.020445049
1 -0.342027186 -0.440126701 -0.146331013 0.389546002 0.039583826 0.349649631 -0.394378390 -0.174293080
2 -0.275102883 -0.249935011 -0.390189124 -0.168028231 -0.497732248 0.469688973 0.383268965 0.408602470
3 -0.371610622 -0.286233664 -0.344583131 0.139393294 -0.106305614 -0.198629076 0.284151301 0.205685996
4 0.267303076 -0.337343770 0.111234003 -0.266131931 -0.232302798 -0.252681226 -0.467706269 -0.029767119
5 -0.044589875 -0.337710840 0.391223159 0.059939654 0.496568630 0.117227069 0.405671778 -0.372225124
6 -0.084505120 -0.323533844 0.328288743 -0.228236587 -0.089922550 0.099282587 0.450314046 -0.194585783
Copyright © Lakebolt ™ Research


lambda
DAλS
KRAKÓW | POLAND
JAVIER
LOPEZ
SPEAKER
lambda
DAλS
KRAKÓW | POLAND

Towards Wide-Spectrum Spreadsheet Computing

Publisher: IEEE Cite This PDF

Enzo Alda ; Javier López Lombano ; Mónica Figuera ; Juan Andrés Escalante ; Richard Lares Mejías ; Pablo Maldonado All Authors

Abstract

Document Sections

Abstract:
with the recent additions of dynamic arrays and user-defined functions, Microsoft Research has taken two important steps towards the unification of spreadsheets and general purpose programming environments. We offer our perspective on that quest, reiterating that the full potential of the spreadsheet computing paradigm can only be realized after removing the entanglement of the computational model and its visualization. Our experimental findings suggest this can be done without detracting from the live and user-friendly nature of the spreadsheet programming experience.

Published in: 2021 4th International Conference on Information and Computer Technologies (ICICT)

Date of Conference: 11-14 March 2021

DOI: 10.1109/ICICT52872.2021.00046

Date Added to IEEE *Xplore*: 15 July 2021

Publisher: IEEE


2024
Ai Creations

Expectativas del Estudiante

- Conocimiento basico de Haskell (recursion, IO basico, pattern matching, funciones de orden superior).
- Estudien el codigo: motivacion y filosofia detras de este.
- Lean la literatura detras de cada tema.
- Un tema por semana idealmente.
- Evaluacion practica:
  - Tareas Individuales 70%: 5 tareas.
  - Proyecto en Pareja 30%: con presentacion, demostracion, y componente teorico (en que paper se baso) detras de el. Deberan redactar una propuesta en PDF para el Lunes de Semana 10, y las exposiciones se realizaran en Semana 12.

Expectativas del Estudiante

- Lean La documentacion. Cada paquete/libreria es una mina de oro de informacion.
- Fuentes informales de informacion:
  - Hoogle: buscar funciones por nombre o por tipo.
  - r/Haskell: subreddit oficial de haskell, profesionales y conferencistas responden preguntas por aca.
  - Haskell Discourse: lo mismo que r/Haskell.
  - r/ProgrammingLanguages: La comunidad expone temas interesantes, papers relevantes.
  - Google Scholar: Google Para Papers.
  - ACM Digital Library: Lo mismo que Google Scholar.

Ambiente de trabajo

- VSCode es facil de configurar y popular, NVim con haskell-tools.
- git(hub): Obligatorio, el curso sube los materiales a un repo publico.
- Haskell:
  - En el repo del curso podran ver la version del compilador y la version del snapshot (stack). De igual forma...
  - GHC 9.12.2
  - snapshot: nightly-2025-11-15
  - Stack 3.9.1
- Nix (Opcional): si poseen un ambiente de Nix el repo provee un flakes con todas las herramientas.
- HLS (Haskell Language Server): si desean tener colores, autocompletado, etc. Van a necesitar tenerlo compilado con GHC 9.12.2.

Map, Filter, Fold

in Lilly

Map

- :: fmapz := fn(functor, seq) -> if(seq = [], [], cons(functor(head(seq)), fmapz(functor, tail(seq))));

Filter

- :: filterz := fn(pred, seq) -> if(pred(head(seq)), cons(pred(head(seq)), filterz(pred, tail(seq))), filterz(pred, tail(seq))));

Fold

- :: foldlz := fn(fn2, init, seq) -> if(seq = [], init, foldlz(fn2, fn2(init, head(seq)), tail(seq))); // fold left
- :: foldrz := fn(fn2, init, seq) -> if(seq = [], init, fn2(head(seq), foldrz(fn2, init, tail(seq)))); // fold right